FNBC y desnormalización controlada en Food Store

Trabajo práctico - Unidad 4
Base de Datos II

Integrantes	Entorno y fecha
Exequiel Arancibia **Joaquin Romero**	PostgreSQL 17.11 America/Buenos_Aires - 20/10/2026

Resolución sobre el proyecto heredado

Se trabaja sobre una copia restaurada del Food Store presentado en los TP1 a TP5. Se mantienen sus cinco tablas del negocio, claves, restricciones, disparadores, índices y vistas. La extensión mayorista de la Parte 1 reutiliza los usuarios existentes; la Parte 2 consulta el mismo esquema **public** del proyecto.

Para medir el reporte del día se agrega una carga de prueba identificada: **25.000 pedidos y 100.000 detalles**. Los datos históricos heredados se conservan y se verifican antes y después de las pruebas. Las mediciones y las salidas de este informe proceden de ejecuciones reales sobre esa copia.

Resultado verificado

ControlLoteAlmacen no cumple FNBC: se descompone sin pérdida y se conserva la regla de unicidad mediante una validación adicional. Para el panel se implementa una vista materializada diaria con sincronización en la misma transacción de escritura y auditoría contra las cuatro tablas fuente.

Contenido	Desarrollo y evidencia
Parte 1 - puntos a-f	Dependencias, clausuras, claves, anomalías, descomposición y migración.
Parte 2 - puntos a-e	Consulta adaptada al esquema real, planes, tiempos y auditoría.
Anexos A, B y C	SQL de ambas partes y de la actividad agregada para el día.

Entrega principal: este PDF contiene el desarrollo completo y los scripts. El paquete complementario conserva además el respaldo, los archivos heredados necesarios y las evidencias para reconstruir y verificar la solución.

TRAZABILIDAD Y COMPATIBILIDAD

Continuidad con los TP1 a TP5

Antecedente entregado	Uso en esta resolución
TP1 - modelo Food Store	Se conservan categoria, producto, usuario, pedido y detalle_pedido; sus PK/FK y subtotal calculado.
TP2 - integridad y concurrencia	Se mantienen las transiciones de estado, la edición de detalles solo en PENDIENTE y el cálculo de pedido.total.
TP3 - carga e índices	El respaldo conserva la carga masiva y los índices aceptados. No se recrea un esquema mínimo alternativo.
TP4 - respaldo e índice analítico	Se restaura respaldo_pre_tp4.dump y se incorpora idx_tp4_detalle_producto_cobertura.
TP5 - índices y vistas	Se incorporan los tres índices aceptados y las vistas originales, incluida mv_facturacion_categoria_mes.

Base de partida y actividad del día

Tabla	Filas heredadas	Filas luego de la carga U4
categoria	2	2
producto	50.000	50.000
usuario	20.000	20.000
pedido	200.000	225.000
detalle_pedido	399.997	499.997

El respaldo proviene del paquete del TP4 y fue generado con PostgreSQL 16.2; esta copia se restauró y midió con PostgreSQL 17.11. Su SHA-256 coincide con el original incluido en los antecedentes. La preparación agrega actividad actual para que el filtro del día produzca un caso medible. La comparación de huellas de filas completas confirma que los registros heredados conservan sus valores y cantidades; se incluye en las evidencias.

Adecuación de 5.1 al modelo heredado

detalle_pedido no tiene eliminado. Se omite solo ese filtro inexistente; se mantiene ped.eliminado = FALSE. **pedido.fecha es TIMESTAMPTZ.** El día se expresa como un intervalo desde la medianoche local inclusive hasta la siguiente exclusive, con zona America/Buenos_Aires. **subtotal es GENERATED STORED**, por cantidad × precio_unitario; se modifican sus componentes, sin escribir el subtotal directamente.

La PK de detalle es (pedido_id,producto_id) y categoria.nombre sigue siendo UNIQUE. El catálogo conserva dos categorías: LIMIT 5 devuelve hasta dos filas. La guía solo filtra pedidos por eliminado, por lo que también cuenta los PENDIENTE; no se agrega la exclusión de CANCELADO propia del reporte mensual del TP5.

PARTE 1 · 4.2(A) Y 4.2(B)

Dependencias y claves candidatas

a) Traducción formal de las reglas de negocio

La extensión se crea en public sobre la copia heredada. Los responsables 801, 802 y 803 ya existen en usuario; solo se agregan los maestros lote y deposito y las relaciones de control.

Se define U = {LoteID, DepositoID, ResponsableControlID}. Para abreviar, L = LoteID, D = DepositoID y R = ResponsableControlID. Las dependencias funcionales son:

Regla de negocio	Dependencia funcional
Un lote y un depósito determinan un único responsable.	{LoteID, DepositoID} → ResponsableControlID **LD → R**
Cada responsable pertenece a un único depósito.	ResponsableControlID → DepositoID **R → D**

El conjunto generador es F = {LD → R, R → D}. No se deduce D → R: un depósito puede tener varios responsables. Tampoco L → D ni L → R: un lote puede distribuirse entre depósitos. Las dependencias se obtienen de las reglas del dominio; una coincidencia en las tres filas de ejemplo no agrega una dependencia válida para todas las instancias.

b) Cálculo de todas las clausuras

X	X^+ respecto de F	Aplicación de dependencias
{ }	{ }	No hay determinantes vacíos.
{L}	{L}	Ninguna regla agrega atributos.
{D}	{D}	Ninguna regla agrega atributos.
{R}	{R, D}	R → D.
{L, D}	{L, D, R}	LD → R.
{L, R}	{L, R, D}	R → D.
{D, R}	{D, R}	No se obtiene L.
{L, D, R}	{L, D, R}	Ya contiene U.

Claves candidatas: {L, D} y {L, R}. Sus clausuras contienen U y ninguna es reducible: L, D y R por separado no determinan todos los atributos. Son todas las claves posibles porque L no aparece a la derecha de ninguna dependencia y, por tanto, debe integrar toda clave; junto con L basta agregar D o R. {L,D,R} es superclave, pero no es mínima.

Atributos primos: LoteID, DepositoID y ResponsableControlID, pues cada uno pertenece a alguna clave candidata. **Atributos no primos:** ninguno. La PK declarada es LD; LR es la otra clave candidata del modelo lógico.

PARTE 1 · 4.2(C) Y 4.2(D)

Violación de FNBC y anomalías

c) Aplicación de la definición formal de FNBC

Una relación está en FNBC si, para toda dependencia no trivial X → Y de F^+, X es superclave. LD → R cumple la condición porque $(LD)^+$ = U. En cambio, **R → D es no trivial y su determinante no es superclave**: R^+ = {R,D}, por lo que falta L. Esta dependencia basta para demostrar que control_lote_almacen **no está en FNBC**.

Que D sea primo no cambia este resultado: la definición de FNBC exige que el determinante sea superclave. Ese detalle permite satisfacer 3FN con estas dependencias, pero no permite concluir FNBC. Tampoco alcanza con mirar la PK: deben considerarse todas las claves y dependencias.

d) Tres anomalías sobre la instancia provista

Inserción. Se quiere registrar que el nuevo responsable 803 pertenece al depósito 30, aunque todavía no tenga controles asignados. La tabla original exige lote_id y utiliza (lote_id, deposito_id) como PK. Por ello no permite guardar ese dato maestro sin inventar un lote o una asignación. La prueba intenta (NULL,30,803), recibe la violación NOT NULL 23502 y revierte la operación.

Borrado. Si se elimina la fila (503,31,802) porque se retira el control del lote 503, desaparece el único registro que vincula al responsable 802 con el depósito 31. Aunque la fila de usuario 802 continúe existiendo, la relación original ya no conserva su depósito de pertenencia. La prueba borra esa fila dentro de una transacción, verifica que no queda ninguna asociación para 802 y ejecuta ROLLBACK.

Actualización. La pertenencia de 801 al depósito 30 está repetida en (501,30,801) y (502,30,801). Si se corrige su depósito a 31 en la primera fila y se omite la segunda, 801 queda asociado a 31 y a 30. La PK no rechaza esa modificación, pero R → D queda incumplida. La prueba cambia solo el lote 501, detecta dos depósitos distintos para 801 y revierte la transacción.

```
-- Detección de la inconsistencia del escenario de actualización.
SELECT responsable_control_id,
       COUNT(DISTINCT deposito_id) AS depositos
FROM public.control_lote_almacen_original
GROUP BY responsable_control_id
HAVING COUNT(DISTINCT deposito_id) > 1;
-- Dentro de la prueba: 801 | 2. Después de ROLLBACK: 0 filas.
```

Las pruebas se ejecutaron sobre la copia original conservada como evidencia. Los cambios de las anomalías se revierten; no alteran la instancia final migrada.

PARTE 1 · 4.2(E)

Descomposición y conservación de las reglas

e) Algoritmo de descomposición sin pérdida

Se toma la dependencia violatoria X → Y, con X = {R}, Y = {D}. El algoritmo reemplaza U por U_1 = {R,D}, la unión de X e Y, y U_2 = U - (Y - X) = {L,R}.

Tabla resultante	Claves y referencias
responsable_deposito(R,D)	PK: R. FK R → public.usuario(id), ya existente. FK D → deposito(id).
lote_responsable(L,R)	PK: (L,R). FK L → lote(id). FK R → responsable_deposito(R).

En ResponsableDeposito, la única dependencia no trivial proyectada es R → D y R es clave. En LoteResponsable no hay dependencias no triviales con un determinante menor que LR. En consecuencia, ambas relaciones están en FNBC. La vista de compatibilidad reconstruye el esquema original con una reunión natural por R, único atributo con el mismo nombre en ambas tablas:

```
CREATE VIEW public.control_lote_almacen AS
SELECT lote_id, deposito_id, responsable_control_id
FROM public.lote_responsable
NATURAL JOIN public.responsable_deposito;
```

Regla que no queda preservada mediante las FK locales

La unión sin pérdida y la preservación de dependencias son propiedades diferentes. Las proyecciones R → D y las PK/FK no imponen por sí mismas LD → R. Por ejemplo, RD = {(801,30),(803,30)} y LR = {(501,801),(501,803)} cumplen esas restricciones locales, pero la vista mostraría dos responsables para el lote 501 y el depósito 30. No se puede declarar UNIQUE(lote_id,deposito_id) en LR porque D no pertenece a esa tabla.

El script completo del Anexo A conserva la regla mediante una validación del JOIN posterior a cada sentencia que modifica cualquiera de las dos tablas. Antes de escribir, un disparador actualiza una fila técnica de guardia, bloqueándola hasta COMMIT o ROLLBACK. Así se serializan las escrituras y se evita que dos sesiones acepten por separado una combinación inválida. Si la validación encuentra COUNT(*) > 1 para un par LD, genera el error 23505 y revierte la sentencia. [3]

La guardia no guarda atributos del negocio y no altera la FNBC de las dos relaciones. Su costo es serializar sus escrituras: se privilegia una implementación didáctica correcta. En aislamientos REPEATABLE READ/SERIALIZABLE una colisión puede exigir reintentar la transacción. Las pruebas rechazaron tanto un segundo responsable para el mismo LD como un cambio de depósito maestro que generaba ese conflicto.

PARTE 1 · 4.2(F)

Demostración y migración verificada

f) Por qué la reunión es sin pérdida

La intersección de U_1 = {R,D} y U_2 = {L,R} es {R}. El criterio para una descomposición binaria exige que esa intersección determine todos los atributos de alguna de las partes. Aquí R → RD por reflexividad y por R → D, de modo que R es superclave de U_1. **La descomposición es sin pérdida respecto de F.** Al reunir las proyecciones de una instancia válida no se pierde ninguna fila original ni aparecen combinaciones espurias.

Migración paso a paso

1. Se carga el esquema original con las tres filas de la guía. 2. Se verifica que un mismo responsable no figure en depósitos distintos. 3. Se conserva una copia estática como control_lote_almacen_original. 4. Se proyectan los maestros RD con DISTINCT y las asignaciones LR. 5. Se crea la vista con el nombre operativo original. 6. Se compara la vista contra la copia en ambos sentidos. Todo el proceso ocurre dentro de una transacción.

```
INSERT INTO public.responsable_deposito
SELECT DISTINCT responsable_control_id, deposito_id
FROM public.control_lote_almacen_original;

INSERT INTO public.lote_responsable
SELECT lote_id, responsable_control_id
FROM public.control_lote_almacen_original;
```

responsable_control_id	deposito_id
801	30
802	31

lote_id	responsable_control_id
501	801
502	801
503	802

Resultado de la vista: (501,30,801), (502,30,801), (503,31,802). **Diferencias bilaterales:** 0. Se utiliza EXCEPT ALL en ambos sentidos para verificar valores y multiplicidades. Las auditorías de LD → R y R → D devuelven cero filas. El script además interrumpe la migración si la equivalencia no se cumple.

La prueba con dos sesiones confirmó la protección de LD → R: una asignación válida se confirma mientras la segunda sesión espera; al continuar, la segunda asignación conflictiva se rechaza con 23505. Luego se retiran las filas de prueba y quedan las tres filas originales. Los registros completos se conservan en las evidencias del paquete.

PARTE 2 - 5.2(A)

Medición de la consulta normalizada

a) Instancia y procedimiento de medición

La copia medida contiene 2 categorías, 50.000 productos, 20.000 usuarios, 225.000 pedidos y 499.997 detalles. La carga del Anexo C inserta pedidos PENDIENTE respetando las restricciones del TP2. Para cargar por lotes se pausa solo trg_recalcular_total, se calculan los totales y se reactiva dentro de la misma transacción. La validación de detalles PENDIENTE permanece activa; luego las escrituras habituales conservan el recálculo heredado.

El día medido reúne **25.000 pedidos no eliminados y 100.000 detalles de esos pedidos**. Se conservan los índices de los antecedentes y se actualizan estadísticas con ANALYZE.

Se ejecuta EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON), con una ejecución de calentamiento y 5 repeticiones medidas por consulta. Se utiliza caché caliente, sin vaciarla entre pruebas. Los parámetros son iguales antes y después: max_parallel_workers_per_gather = 0, jit = off, work_mem = 4MB y shared_buffers = 128MB. Execution Time excluye el arranque de psql y la planificación. [1]

Consulta de la guía adaptada al esquema heredado

```
SET TIME ZONE 'America/Buenos_Aires';
EXPLAIN (ANALYZE, BUFFERS)
SELECT c.nombre AS categoria,SUM(dp.subtotal) AS total_vendido
FROM public.detalle_pedido dp
JOIN public.producto pr ON pr.id=dp.producto_id
JOIN public.categoria c ON c.id=pr.categoria_id
JOIN public.pedido ped ON ped.id=dp.pedido_id
WHERE ped.fecha >= CURRENT_DATE::timestamptz
AND ped.fecha < (CURRENT_DATE+1)::timestamptz AND ped.eliminado=FALSE
GROUP BY c.nombre
ORDER BY total_vendido DESC
LIMIT 5;
```

Tiempo mediano: 173,044 ms. Las ejecuciones medidas fueron 170,856, 173,790, 171,557, 173,663, 173,044 ms. La captura textual adicional registró 176,807 ms y corresponde a otra ejecución real.

Nodo dominante por trabajo propio aproximado: Hash Join. En la ejecución mediana presenta 49,309 ms de trabajo propio aproximado, 100.000 filas por ciclo y 1 ciclo(s). Su costo estimado total es 18794.97 unidades. Los planes completos muestran los recorridos, reuniones y agregación necesarios antes de producir el reporte.

Los tiempos de los nodos superiores incluyen a sus hijos; se resta el tiempo de los hijos inmediatos para localizar aproximadamente el trabajo propio, sin ejecución paralela. Las unidades de cost no son milisegundos. [1]

PARTE 2 - 5.2(B) Y 5.2(C)

Patrón elegido e implementación

b) Justificación del patrón (un párrafo)

Se elige una **vista materializada por día local y nombre de categoría**: el SELECT normalizado registra una mediana de 173,044 ms y su nodo dominante es Hash Join; repetir ese trabajo en cada lectura del panel motiva almacenar el agregado. La sincronización se garantiza con disparadores de sentencia sobre las cuatro fuentes: un candado transaccional serializa escritores y el refresco se ejecuta en la misma transacción, en READ COMMITTED, respetando el recálculo heredado de totales. La solución es reversible sin pérdida de información porque conserva las tablas originales como fuente de verdad; retirar los objetos U4 permite volver al SELECT normalizado. [2][3]

c) Estructura diaria independiente de la vista mensual

```
CREATE MATERIALIZED VIEW public.mv_u4_ventas_categoria_dia AS
SELECT (ped.fecha AT TIME ZONE
        'America/Argentina/Buenos_Aires')::date AS fecha,
       c.nombre AS categoria, SUM(dp.subtotal) AS total_vendido
FROM public.detalle_pedido dp
JOIN public.producto pr ON pr.id = dp.producto_id
JOIN public.categoria c ON c.id = pr.categoria_id
JOIN public.pedido ped ON ped.id = dp.pedido_id
WHERE ped.eliminado = FALSE
GROUP BY (ped.fecha AT TIME ZONE
          'America/Argentina/Buenos_Aires')::date, c.nombre;

CREATE INDEX idx_u4_ventas_categoria_dia_top
ON public.mv_u4_ventas_categoria_dia(fecha, total_vendido DESC);
```

Se resumen todas las fechas: el cambio de día no deja materializado únicamente el día anterior. La fecha del resumen usa explícitamente la zona del negocio. Se agrupa por nombre para reproducir la consulta de la guía, aunque el esquema heredado además impide nombres duplicados.

La instalación bloquea las cuatro fuentes durante el alta para no perder escrituras. Los disparadores BEFORE obtienen el mismo candado y rechazan aislamientos distintos de READ COMMITTED. El AFTER refresca el resumen por sentencia; en los UPDATE internos de pedido que dispara recalcular_total_pedido se omite el refresco anidado. El AFTER exterior de detalle_pedido refresca una vez, cuando terminaron todas las líneas y sus totales. [3][4]

TRUNCATE se rechaza con 23514 para preservar los totales y las reglas heredadas. Los cambios confirmados y el resumen quedan en la misma transacción. mv_facturacion_categoria_mes del TP5 conserva su definición: se refresca explícitamente tras la carga inicial y mantiene su política de snapshot para cambios posteriores. El panel de este TP usa la nueva vista diaria. El Anexo B incluye funciones, disparadores, consulta y auditoría completos.

PARTE 2 - 5.2(D)

Consulta optimizada y comparación

d) Lectura directa del resumen

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT categoria,total_vendido
FROM public.mv_u4_ventas_categoria_dia
WHERE fecha=CURRENT_DATE
ORDER BY total_vendido DESC
LIMIT 5;
```

El panel lee únicamente la nueva vista materializada; no vuelve a recorrer y unir pedido, detalle_pedido, producto y categoria. El plan real permite comprobar el método de acceso elegido por el optimizador para el resumen y el filtro del día.

Antes - consulta normalizada	Después - vista materializada
Mediana de 5 ejecuciones: 173,044 ms	**Mediana de 5 ejecuciones:** 0,095 ms
Rango: 170,856 a 173,790 ms	**Rango:** 0,091 a 0,111 ms
Nodo dominante: Hash Join.	**Nodo dominante:** Seq Scan sobre mv_u4_ventas_categoria_dia.
Captura textual: 176,807 ms	**Captura textual:** 0,090 ms
Buffers, ejecución mediana: 6.874 hits.	**Buffers, ejecución mediana:** 5 hits.

La mediana disminuye 99.95 %, una razón de 1821.5 entre medianas. Es un resultado local sobre la copia y carga documentadas; debe medirse nuevamente frente a otra distribución de datos o nivel de concurrencia. El costo de mantener la vista se informa por separado.

Igualdad del reporte antes y después

Categoría	Monto del día
Pizzas	137.506.401,00
Bebidas	137.503.654,00

Ambas consultas devolvieron exactamente las mismas 2 filas y montos. LIMIT 5 es un máximo: el proyecto contiene 2 categorías y no se inventan otras para completar cinco puestos. Si hubiera empates, se puede agregar el mismo desempate por categoria a ambas consultas.

PARTE 2 - 5.2(E)

Auditoría e integridad del proyecto

e) Comparación contra la fuente de verdad

El script de auditoría recalcula los montos diarios desde las cuatro fuentes y compara ambos sentidos con EXCEPT ALL. Detecta grupos faltantes, sobrantes y montos distintos en cualquier fecha; no se limita al top del día. El candado común estabiliza la comparación frente a los escritores. También se comprueba que pedido.total coincida con sus detalles, para verificar la integración con TP1 y TP2.

```
 sentido | fecha | categoria | total_vendido
---------+-------+-----------+--------------
(0 rows)
```

Control ejecutado	Resultado real
Auditoría luego de instalar	0 diferencias entre resumen y fuente.
20 escenarios funcionales, con ROLLBACK	Auditorías consistentes; rechazos esperados controlados.
Escrituras concurrentes, con COMMIT	La segunda sesión espera el candado; se comprueban ambos cambios.
Auditoría final	0 diferencias; 0 pedidos con total incorrecto.
Historial heredado	Mismos valores y conteos antes y después, verificados con huellas de filas completas.

Las pruebas abarcan cambios válidos en detalles de pedidos PENDIENTE, bajas y reactivación de pedido, fecha, categoría de producto y operaciones de alta/borrado. Los casos incompatibles con las restricciones heredadas deben rechazarse, incluida la modificación de un detalle no editable y TRUNCATE. El registro JSON conserva el SQL, el estado esperado y la auditoría de cada caso.

La prueba de recursión verifica una sentencia que modifica cuatro líneas: se omiten los cuatro refrescos anidados provocados por el recálculo de pedido.total y se ejecuta un único refresco al terminar la sentencia exterior. No se desactivan los disparadores de integridad heredados.

La prueba concurrente utiliza cambios reales: la primera sesión modifica una línea y la segunda espera el candado antes de modificar otra. Después de ambos COMMIT se audita el resumen y se verifica la incorporación de ambos cambios; luego se restauran los valores y se audita nuevamente. La igualdad final del top confirma que las pruebas no dejan alterada la carga de medición.

Costo de la sincronización inmediata

El refresco reconstruye el resumen completo. Una escritura de detalle medida dentro de BEGIN/ROLLBACK registró **922,234 ms en el refresco** y 930,009 ms de ejecución total, incluidos los disparadores. Se paga ese costo por sentencia exterior de escritura y se serializan los escritores; REFRESH sin CONCURRENTLY también puede bloquear lectores. [2]

Esta política prioriza que una escritura confirmada deje el panel actualizado. Para una carga con muchas escrituras corresponde medir mantenimiento incremental u otra estrategia compatible con la frescura requerida; la mejora del SELECT por sí sola no prueba una mejora de todo el sistema.

EVIDENCIA - EXPLAIN ANALYZE ANTES

Plan real de la consulta normalizada

Imagen de la salida real de psql. Solo se ajusta la presentación de líneas; el paquete conserva el TXT. Esta ejecución adicional no integra las repeticiones usadas para la mediana.

EXPLAIN ANALYZE - consulta normalizada

Salida real de psql | PostgreSQL 17.11 | 20/10/2026

```
Limit  (cost=21566.94..21566.94 rows=2 width=170) (actual time=176.183..176.187 rows=2 loops=1)
  Buffers: shared hit=6874
  ->  Sort  (cost=21566.94..21566.94 rows=2 width=170) (actual time=176.180..176.184 rows=2 loops=1)
        Sort Key: (sum(dp.subtotal)) DESC
        Sort Method: quicksort  Memory: 25kB
        Buffers: shared hit=6874
        ->  HashAggregate  (cost=21566.90..21566.93 rows=2 width=170) (actual time=176.152..176.155 rows=2
           loops=1)
              Group Key: c.nombre
              Batches: 1  Memory Usage: 24kB
              Buffers: shared hit=6871
              ->  Hash Join  (cost=10787.53..21317.14 rows=49953 width=143) (actual time=99.294..157.527
                 rows=100000 loops=1)
                    Hash Cond: (pr.categoria_id = c.id)
                    Buffers: shared hit=6871
                    ->  Hash Join  (cost=10786.49..20907.11 rows=49953 width=13) (actual time=99.266..144.406
                       rows=100000 loops=1)
                          Hash Cond: (dp.producto_id = pr.id)
                          Buffers: shared hit=6870
                          ->  Hash Join  (cost=8805.49..18794.97 rows=49953 width=13) (actual
                             time=87.872..114.278 rows=100000 loops=1)
                                Hash Cond: (dp.pedido_id = ped.id)
                                Buffers: shared hit=6014
                                ->  Seq Scan on detalle_pedido dp  (cost=0.00..8676.97 rows=499997 width=21)
                                   (actual time=0.020..29.592 rows=499997 loops=1)
                                      Buffers: shared hit=3677
                                ->  Hash  (cost=8524.50..8524.50 rows=22479 width=8) (actual time=32.585..32.586
                                   rows=25000 loops=1)
                                      Buckets: 32768  Batches: 1  Memory Usage: 1233kB
                                      Buffers: shared hit=2337
                                      ->  Seq Scan on pedido ped  (cost=0.00..8524.50 rows=22479 width=8)
                                         (actual time=24.968..30.082 rows=25000 loops=1)
                                            Filter: ((NOT eliminado) AND (fecha >= (CURRENT_DATE)::timestamp
                                              with time zone) AND (fecha < ((CURRENT_DATE + 1))::timestamp with
                                              time zone))
                                            Rows Removed by Filter: 200000
                                            Buffers: shared hit=2337
                          ->  Hash  (cost=1356.00..1356.00 rows=50000 width=16) (actual time=11.266..11.266
                             rows=50000 loops=1)
                                Buckets: 65536  Batches: 1  Memory Usage: 2856kB
                                Buffers: shared hit=856
                                ->  Seq Scan on producto pr  (cost=0.00..1356.00 rows=50000 width=16) (actual
                                   time=0.008..6.512 rows=50000 loops=1)
                                      Buffers: shared hit=856
                    ->  Hash  (cost=1.02..1.02 rows=2 width=146) (actual time=0.019..0.020 rows=2 loops=1)
                          Buckets: 1024  Batches: 1  Memory Usage: 9kB
                          Buffers: shared hit=1
                          ->  Seq Scan on categoria c  (cost=0.00..1.02 rows=2 width=146) (actual
                             time=0.013..0.013 rows=2 loops=1)
                                Buffers: shared hit=1
Planning:
  Buffers: shared hit=514
Planning Time: 9.856 ms
Execution Time: 176.807 ms
```

EVIDENCIA - EXPLAIN ANALYZE DESPUÉS

Lectura del resumen y mantenimiento

Salidas reales adicionales a los planes JSON de las mediciones principales. La escritura se ejecuta en BEGIN/ROLLBACK.

EXPLAIN ANALYZE - resumen diario U4

Salida real de psql | PostgreSQL 17.11 | 20/10/2026

```
Limit  (cost=6.30..6.30 rows=2 width=13) (actual time=0.060..0.060 rows=2 loops=1)
  Buffers: shared hit=5
  ->  Sort  (cost=6.30..6.30 rows=2 width=13) (actual time=0.058..0.058 rows=2 loops=1)
        Sort Key: total_vendido DESC
        Sort Method: quicksort  Memory: 25kB
        Buffers: shared hit=5
        ->  Seq Scan on mv_u4_ventas_categoria_dia  (cost=0.00..6.29 rows=2 width=13) (actual time=0.021..0.031
           rows=2 loops=1)
              Filter: (fecha = CURRENT_DATE)
              Rows Removed by Filter: 284
              Buffers: shared hit=2
Planning:
  Buffers: shared hit=79
Planning Time: 1.721 ms
Execution Time: 0.090 ms
```

EXPLAIN ANALYZE - costo del mantenimiento

Salida real de psql | PostgreSQL 17.11 | 20/10/2026

```
Update on detalle_pedido  (cost=0.42..8.44 rows=0 width=0) (actual time=4.713..4.713 rows=0 loops=1)
  Buffers: shared hit=485
  ->  Index Scan using idx_u3_detalle_producto_pedido on detalle_pedido  (cost=0.42..8.44 rows=1 width=11)
     (actual time=0.037..0.044 rows=1 loops=1)
        Index Cond: ((producto_id = 18) AND (pedido_id = 200001))
        Buffers: shared hit=9
Planning:
  Buffers: shared hit=120
Planning Time: 2.511 ms
Trigger trg_recalcular_total: time=0.801 calls=1
Trigger trg_validar_detalle_pendiente: time=3.272 calls=1
Trigger ventas_bloqueo: time=1.044 calls=1
Trigger ventas_refresco: time=930.111 calls=1
Execution Time: 935.817 ms
```

Control final automatizado

```
difference_count = 0
incorrect_order_total_count = 0
Auditoría de diferencias: (0 rows)
```

GUÍA DE EJECUCIÓN

Reconstrucción y referencias

Cómo reproducir esta entrega

El paquete conserva el respaldo heredado y los scripts originales de índices y vistas necesarios para reconstruir el punto de partida. 00_reconstruir_base.ps1 crea la base de práctica, restaura el respaldo, aplica los índices y vistas previos y agrega la actividad del día. Después se ejecutan los dos scripts principales U4. Revisar los parámetros de conexión PostgreSQL; los scripts verifican que la base sea foodstore_u4_arancibia_romero.

```
1. Restaurar respaldo_pre_tp4.dump en la base de práctica.
2. Aplicar índice TP4 y los índices/vistas originales TP5.
3. Ejecutar 01_actividad_del_dia.sql.
4. Ejecutar tp_fnbc_control_lote.sql.
5. Ejecutar tp_desnormalizacion_top_categorias.sql.
6. Ejecutar las verificaciones de reproduccion/.
Usar ON_ERROR_STOP=1 y no restaurar sobre la base de trabajo original.
```

El Anexo C agrega actividad relativa a CURRENT_DATE en la zona del negocio. Si se reconstruye otro día, esas filas se ubican en el nuevo día de prueba. Las fechas y cifras medidas aquí corresponden a la ejecución documentada; se deben volver a medir después de reconstruir. El respaldo contiene los datos heredados y no es necesario volver a ejecutar los scripts antiguos que eliminan/recrean tablas.

Reversibilidad

La Parte 2 mantiene pedido, detalle_pedido, producto y categoria como fuente de verdad. Dentro de una transacción se bloquean las fuentes como en la instalación y se retiran únicamente los disparadores ventas_bloqueo/ventas_refresco, las funciones u4_serializar_ventas()/u4_refrescar_ventas(), el índice y mv_u4_ventas_categoria_dia. Las reglas del TP2, el cálculo de pedido.total y las vistas del TP5 permanecen. El panel puede volver al SELECT normalizado.

Fuentes del trabajo y documentación

Antecedentes proporcionados: TP1 (modelo y esquema), TP2 (restricciones), TP3 (carga e índices), TP4 (respaldo y optimización) y TP5 (índices y vistas), de Exequiel Arancibia y Joaquin Romero. Sus archivos necesarios y procedencia se conservan dentro de antecedentes/ del paquete. Consigna actual: guía Unidad 4, puntos 4.2(a-f), 5.2(a-e) y entregables. La plataforma solicita un único PDF; por eso se incorporan los scripts en los anexos.

[1] PostgreSQL 17. EXPLAIN: costos y tiempos.

[2] PostgreSQL 17. REFRESH MATERIALIZED VIEW.

[3] PostgreSQL 17. Funciones VOLATILE y visibilidad.

[4] PostgreSQL 17. Aislamiento y READ COMMITTED.

ANEXO A - TP_FNBC_CONTROL_LOTE.SQL

FNBC: esquema y migración

Código completo: solo se omiten comentarios y líneas vacías para facilitar la lectura. El paquete conserva los archivos .sql originales, incluidos sus comentarios.

```
BEGIN;
SET LOCAL search_path = pg_catalog, public;
DO $$
BEGIN
    IF current_database() <> 'foodstore_u4_arancibia_romero' THEN
        RAISE EXCEPTION 'Base incorrecta: se esperaba foodstore_u4_arancibia_romero, se obtuvo %',
            current_database();
    END IF;
    IF to_regclass('public.usuario') IS NULL THEN
        RAISE EXCEPTION 'Falta public.usuario: primero restaurar la base acumulativa TP1-TP5';
    END IF;
    IF (SELECT COUNT(*) FROM public.usuario WHERE id IN (801, 802, 803)) <> 3 THEN
        RAISE EXCEPTION 'La copia debe conservar los usuarios existentes 801, 802 y 803';
    END IF;
END
$$;
CREATE TABLE public.lote (
    id BIGINT PRIMARY KEY,
    codigo VARCHAR(40) NOT NULL UNIQUE
);
CREATE TABLE public.deposito (
    id BIGINT PRIMARY KEY,
    nombre VARCHAR(120) NOT NULL UNIQUE
);
INSERT INTO public.lote (id, codigo) VALUES
    (501, 'LOTE-U4-501'),
    (502, 'LOTE-U4-502'),
    (503, 'LOTE-U4-503');
INSERT INTO public.deposito (id, nombre) VALUES
    (30, 'Deposito Central Food Store'),
    (31, 'Deposito Norte Food Store');
CREATE TABLE public.control_lote_almacen (
    lote_id BIGINT NOT NULL REFERENCES public.lote(id) ON DELETE RESTRICT,
    deposito_id BIGINT NOT NULL REFERENCES public.deposito(id) ON DELETE RESTRICT,
    responsable_control_id BIGINT NOT NULL REFERENCES public.usuario(id) ON DELETE RESTRICT,
    PRIMARY KEY (lote_id, deposito_id)
);
INSERT INTO public.control_lote_almacen
    (lote_id, deposito_id, responsable_control_id)
VALUES
    (501, 30, 801),
    (502, 30, 801),
    (503, 31, 802);
DO $$
BEGIN
    IF EXISTS (
        SELECT responsable_control_id
        FROM public.control_lote_almacen
        GROUP BY responsable_control_id
        HAVING COUNT(DISTINCT deposito_id) > 1
    ) THEN
        RAISE EXCEPTION USING
            ERRCODE = '23514',
            MESSAGE = 'La instancia original incumple ResponsableControlID -> DepositoID';
    END IF;
END
$$;
ALTER TABLE public.control_lote_almacen
    RENAME TO control_lote_almacen_original;
CREATE TABLE public.responsable_deposito (
    responsable_control_id BIGINT PRIMARY KEY
        REFERENCES public.usuario(id) ON DELETE RESTRICT,
    deposito_id BIGINT NOT NULL
        REFERENCES public.deposito(id) ON DELETE RESTRICT
);
CREATE TABLE public.lote_responsable (
    lote_id BIGINT NOT NULL
        REFERENCES public.lote(id) ON DELETE RESTRICT,
    responsable_control_id BIGINT NOT NULL
        REFERENCES public.responsable_deposito(responsable_control_id) ON DELETE RESTRICT,
    PRIMARY KEY (lote_id, responsable_control_id)
);
INSERT INTO public.responsable_deposito
    (responsable_control_id, deposito_id)
SELECT DISTINCT responsable_control_id, deposito_id
FROM public.control_lote_almacen_original;
```

```
INSERT INTO public.lote_responsable
    (lote_id, responsable_control_id)
SELECT lote_id, responsable_control_id
FROM public.control_lote_almacen_original;
CREATE VIEW public.control_lote_almacen AS
SELECT lote_id, deposito_id, responsable_control_id
FROM public.lote_responsable
NATURAL JOIN public.responsable_deposito;
CREATE TABLE public.guardia_control_lote (
    id SMALLINT PRIMARY KEY CHECK (id = 1),
    turno BOOLEAN NOT NULL
);
INSERT INTO public.guardia_control_lote (id, turno) VALUES (1, FALSE);
CREATE FUNCTION public.bloquear_control_lote()
RETURNS trigger
LANGUAGE plpgsql
VOLATILE
AS $$
BEGIN
    UPDATE public.guardia_control_lote
    SET turno = NOT turno
    WHERE id = 1;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'Falta la fila de sincronización de control de lotes';
    END IF;
    RETURN NULL;
END
$$;
CREATE FUNCTION public.verificar_control_lote()
RETURNS trigger
LANGUAGE plpgsql
VOLATILE
AS $$
BEGIN
    IF EXISTS (
        SELECT lr.lote_id, rd.deposito_id
        FROM public.lote_responsable AS lr
        JOIN public.responsable_deposito AS rd
            USING (responsable_control_id)
        GROUP BY lr.lote_id, rd.deposito_id
        HAVING COUNT(*) > 1
    ) THEN
        RAISE EXCEPTION USING
            ERRCODE = '23505',
            MESSAGE = 'Ya existe otro responsable para el mismo lote y depósito';
    END IF;
    RETURN NULL;
END
$$;
CREATE TRIGGER bloquear_lote_responsable
BEFORE INSERT OR UPDATE OR DELETE
ON public.lote_responsable
FOR EACH STATEMENT
EXECUTE FUNCTION public.bloquear_control_lote();
CREATE TRIGGER verificar_lote_responsable
AFTER INSERT OR UPDATE OR DELETE
ON public.lote_responsable
FOR EACH STATEMENT
EXECUTE FUNCTION public.verificar_control_lote();
CREATE TRIGGER bloquear_responsable_deposito
BEFORE INSERT OR UPDATE OR DELETE
ON public.responsable_deposito
FOR EACH STATEMENT
EXECUTE FUNCTION public.bloquear_control_lote();
CREATE TRIGGER verificar_responsable_deposito
AFTER INSERT OR UPDATE OR DELETE
ON public.responsable_deposito
FOR EACH STATEMENT
EXECUTE FUNCTION public.verificar_control_lote();
DO $$
DECLARE
    rechazo BOOLEAN := FALSE;
BEGIN
    INSERT INTO public.responsable_deposito
        (responsable_control_id, deposito_id)
    VALUES (803, 30);
    BEGIN
        INSERT INTO public.lote_responsable
            (lote_id, responsable_control_id)
        VALUES (501, 803);
    EXCEPTION
        WHEN unique_violation THEN
            rechazo := TRUE;
    END;
    IF NOT rechazo THEN
        RAISE EXCEPTION 'Prueba fallida: se permitió duplicar un lote y depósito';
```

```
    END IF;
    DELETE FROM public.responsable_deposito
    WHERE responsable_control_id = 803;
    RAISE NOTICE 'Correcto: se rechazó un segundo responsable para L501 y D30';
END
$$;
DO $$
DECLARE
    rechazo BOOLEAN := FALSE;
BEGIN
    INSERT INTO public.responsable_deposito
        (responsable_control_id, deposito_id)
    VALUES (803, 31);
    INSERT INTO public.lote_responsable
        (lote_id, responsable_control_id)
    VALUES (501, 803);
    BEGIN
        UPDATE public.responsable_deposito
        SET deposito_id = 30
        WHERE responsable_control_id = 803;
    EXCEPTION
        WHEN unique_violation THEN
            rechazo := TRUE;
    END;
    IF NOT rechazo THEN
        RAISE EXCEPTION 'Prueba fallida: el cambio maestro permitió duplicar LD';
    END IF;
    DELETE FROM public.lote_responsable
    WHERE lote_id = 501 AND responsable_control_id = 803;
    DELETE FROM public.responsable_deposito
    WHERE responsable_control_id = 803;
    RAISE NOTICE 'Correcto: también se rechazó el cambio maestro que duplicaba LD';
END
$$;
DO $$
BEGIN
    IF EXISTS (
        SELECT lote_id, deposito_id, responsable_control_id
        FROM public.control_lote_almacen_original
        EXCEPT ALL
        SELECT lote_id, deposito_id, responsable_control_id
        FROM public.control_lote_almacen
    ) OR EXISTS (
        SELECT lote_id, deposito_id, responsable_control_id
        FROM public.control_lote_almacen
        EXCEPT ALL
        SELECT lote_id, deposito_id, responsable_control_id
        FROM public.control_lote_almacen_original
    ) THEN
        RAISE EXCEPTION 'La migración no reconstruye la instancia original';
    END IF;
END
$$;
COMMIT;
SELECT COUNT(*) AS diferencias
FROM (
    (
        SELECT lote_id, deposito_id, responsable_control_id
        FROM public.control_lote_almacen_original
        EXCEPT ALL
        SELECT lote_id, deposito_id, responsable_control_id
        FROM public.control_lote_almacen
    )
    UNION ALL
    (
        SELECT lote_id, deposito_id, responsable_control_id
        FROM public.control_lote_almacen
        EXCEPT ALL
        SELECT lote_id, deposito_id, responsable_control_id
        FROM public.control_lote_almacen_original
    )
) AS diferencias_bilaterales;
SELECT lote_id, deposito_id, responsable_control_id
FROM public.control_lote_almacen
ORDER BY lote_id, deposito_id, responsable_control_id;
SELECT lote_id, deposito_id, COUNT(*) AS cantidad_responsables
FROM public.control_lote_almacen
GROUP BY lote_id, deposito_id
HAVING COUNT(*) > 1;
SELECT responsable_control_id, COUNT(DISTINCT deposito_id) AS cantidad_depositos
FROM public.control_lote_almacen
GROUP BY responsable_control_id
HAVING COUNT(DISTINCT deposito_id) > 1;
```

ANEXO B - TP_DESNORMALIZACION_TOP_CATEGORIAS.SQL

Desnormalización, consulta y auditoría

Código completo: solo se omiten comentarios y líneas vacías para facilitar la lectura. El paquete conserva los archivos .sql originales, incluidos sus comentarios.

```
\set ON_ERROR_STOP on
SET timezone = 'America/Argentina/Buenos_Aires';
SET default_transaction_isolation = 'read committed';
SET max_parallel_workers_per_gather = 0;
SET jit = off;
DO $$
BEGIN
    IF current_database() <> 'foodstore_u4_arancibia_romero' THEN
        RAISE EXCEPTION 'Base incorrecta: %. Usar foodstore_u4_arancibia_romero',
            current_database();
    END IF;
    IF to_regclass('public.detalle_pedido') IS NULL
       OR to_regclass('public.mv_facturacion_categoria_mes') IS NULL
       OR NOT EXISTS (
           SELECT 1 FROM pg_trigger
           WHERE tgrelid='public.detalle_pedido'::regclass
             AND tgname='trg_validar_detalle_pendiente' AND NOT tgisinternal
       ) THEN
        RAISE EXCEPTION 'Falta la base heredada acumulativa de TP2 a TP5';
    END IF;
    IF NOT EXISTS (
        SELECT 1 FROM pg_attribute
        WHERE attrelid='public.pedido'::regclass AND attname='fecha'
          AND atttypid='timestamptz'::regtype AND NOT attisdropped
    ) OR NOT EXISTS (
        SELECT 1 FROM pg_attribute
        WHERE attrelid='public.detalle_pedido'::regclass AND attname='subtotal'
          AND attgenerated='s' AND NOT attisdropped
    ) THEN
        RAISE EXCEPTION 'Se esperaba fecha TIMESTAMPTZ y subtotal generado heredados';
    END IF;
END;
$$;
EXPLAIN (ANALYZE, BUFFERS)
SELECT c.nombre AS categoria, SUM(dp.subtotal) AS total_vendido
FROM public.detalle_pedido dp
JOIN public.producto pr ON pr.id=dp.producto_id
JOIN public.categoria c ON c.id=pr.categoria_id
JOIN public.pedido ped ON ped.id=dp.pedido_id
WHERE ped.fecha >= CURRENT_DATE::timestamptz
  AND ped.fecha < (CURRENT_DATE+1)::timestamptz
  AND ped.eliminado=FALSE
GROUP BY c.nombre
ORDER BY total_vendido DESC
LIMIT 5;
BEGIN;
LOCK TABLE public.categoria,public.producto,public.pedido,public.detalle_pedido
IN SHARE ROW EXCLUSIVE MODE;
CREATE MATERIALIZED VIEW public.mv_u4_ventas_categoria_dia AS
SELECT (ped.fecha AT TIME ZONE 'America/Argentina/Buenos_Aires')::date AS fecha,
       c.nombre AS categoria, SUM(dp.subtotal) AS total_vendido
FROM public.detalle_pedido dp
JOIN public.producto pr ON pr.id=dp.producto_id
JOIN public.categoria c ON c.id=pr.categoria_id
JOIN public.pedido ped ON ped.id=dp.pedido_id
WHERE ped.eliminado=FALSE
GROUP BY (ped.fecha AT TIME ZONE 'America/Argentina/Buenos_Aires')::date,c.nombre;
CREATE INDEX idx_u4_ventas_categoria_dia_top
ON public.mv_u4_ventas_categoria_dia(fecha,total_vendido DESC);
CREATE FUNCTION public.u4_serializar_ventas()
RETURNS trigger LANGUAGE plpgsql VOLATILE
SET search_path=pg_catalog AS $$
BEGIN
    IF TG_OP='TRUNCATE' THEN
        RAISE EXCEPTION
            'TRUNCATE no admitido sobre %.%: preservar detalle, total y estados heredados',
            TG_TABLE_SCHEMA,TG_TABLE_NAME
            USING ERRCODE='23514';
    END IF;
    IF current_setting('transaction_isolation') <> 'read committed' THEN
        RAISE EXCEPTION 'La sincronizacion U4 requiere READ COMMITTED'
            USING ERRCODE='25001';
    END IF;
    PERFORM pg_advisory_xact_lock(204620,4);
    RETURN NULL;
END;
```

```
$$;
CREATE FUNCTION public.u4_refrescar_ventas()
RETURNS trigger LANGUAGE plpgsql VOLATILE
SET search_path=pg_catalog AS $$
BEGIN
    IF TG_TABLE_SCHEMA='public' AND TG_TABLE_NAME='pedido'
       AND pg_trigger_depth()>1 THEN
        RAISE DEBUG 'U4_REFRESH_SKIPPED table=% depth=%',TG_TABLE_NAME,pg_trigger_depth();
        RETURN NULL;
    END IF;
    RAISE DEBUG 'U4_REFRESH_EXECUTED table=% depth=%',TG_TABLE_NAME,pg_trigger_depth();
    REFRESH MATERIALIZED VIEW public.mv_u4_ventas_categoria_dia;
    RETURN NULL;
END;
$$;
CREATE TRIGGER ventas_bloqueo
BEFORE INSERT OR UPDATE OR DELETE OR TRUNCATE ON public.detalle_pedido
FOR EACH STATEMENT EXECUTE FUNCTION public.u4_serializar_ventas();
CREATE TRIGGER ventas_refresco
AFTER INSERT OR UPDATE OR DELETE ON public.detalle_pedido
FOR EACH STATEMENT EXECUTE FUNCTION public.u4_refrescar_ventas();
CREATE TRIGGER ventas_bloqueo
BEFORE INSERT OR UPDATE OR DELETE OR TRUNCATE ON public.pedido
FOR EACH STATEMENT EXECUTE FUNCTION public.u4_serializar_ventas();
CREATE TRIGGER ventas_refresco
AFTER INSERT OR UPDATE OR DELETE ON public.pedido
FOR EACH STATEMENT EXECUTE FUNCTION public.u4_refrescar_ventas();
CREATE TRIGGER ventas_bloqueo
BEFORE INSERT OR UPDATE OR DELETE OR TRUNCATE ON public.producto
FOR EACH STATEMENT EXECUTE FUNCTION public.u4_serializar_ventas();
CREATE TRIGGER ventas_refresco
AFTER INSERT OR UPDATE OR DELETE ON public.producto
FOR EACH STATEMENT EXECUTE FUNCTION public.u4_refrescar_ventas();
CREATE TRIGGER ventas_bloqueo
BEFORE INSERT OR UPDATE OR DELETE OR TRUNCATE ON public.categoria
FOR EACH STATEMENT EXECUTE FUNCTION public.u4_serializar_ventas();
CREATE TRIGGER ventas_refresco
AFTER INSERT OR UPDATE OR DELETE ON public.categoria
FOR EACH STATEMENT EXECUTE FUNCTION public.u4_refrescar_ventas();
COMMIT;
ANALYZE public.mv_u4_ventas_categoria_dia;
EXPLAIN (ANALYZE, BUFFERS)
SELECT categoria,total_vendido
FROM public.mv_u4_ventas_categoria_dia
WHERE fecha=CURRENT_DATE
ORDER BY total_vendido DESC
LIMIT 5;
SELECT categoria,total_vendido
FROM public.mv_u4_ventas_categoria_dia
WHERE fecha=CURRENT_DATE
ORDER BY total_vendido DESC
LIMIT 5;
BEGIN ISOLATION LEVEL READ COMMITTED;
DO $$ BEGIN PERFORM pg_advisory_xact_lock(204620,4); END $$;
WITH fuente AS (
    SELECT (ped.fecha AT TIME ZONE 'America/Argentina/Buenos_Aires')::date AS fecha,
           c.nombre AS categoria,SUM(dp.subtotal) AS total_vendido
    FROM public.detalle_pedido dp
    JOIN public.producto pr ON pr.id=dp.producto_id
    JOIN public.categoria c ON c.id=pr.categoria_id
    JOIN public.pedido ped ON ped.id=dp.pedido_id
    WHERE ped.eliminado=FALSE
    GROUP BY (ped.fecha AT TIME ZONE 'America/Argentina/Buenos_Aires')::date,c.nombre
), diferencias AS (
    SELECT 'fuente-mv' AS sentido,d.* FROM (
        SELECT * FROM fuente
        EXCEPT ALL
        SELECT fecha,categoria,total_vendido FROM public.mv_u4_ventas_categoria_dia
    ) d
    UNION ALL
    SELECT 'mv-fuente',d.* FROM (
        SELECT fecha,categoria,total_vendido FROM public.mv_u4_ventas_categoria_dia
        EXCEPT ALL
        SELECT * FROM fuente
    ) d
)
SELECT * FROM diferencias;
COMMIT;
WITH totales AS (
    SELECT pedido_id,SUM(subtotal) AS total FROM public.detalle_pedido GROUP BY pedido_id
)
SELECT p.id,p.total AS almacenado,COALESCE(t.total,0) AS calculado
FROM public.pedido p LEFT JOIN totales t ON t.pedido_id=p.id
WHERE p.total IS DISTINCT FROM COALESCE(t.total,0::numeric);
```

ANEXO C - 01_ACTIVIDAD_DEL_DIA.SQL

Actividad de prueba del día

Código completo: solo se omiten comentarios y líneas vacías para facilitar la lectura. El paquete conserva los archivos .sql originales, incluidos sus comentarios.

```
\set ON_ERROR_STOP on
SET search_path = public, pg_catalog;
SET TIME ZONE 'America/Argentina/Buenos_Aires';
BEGIN;
DO $$
BEGIN
    IF current_database() <> 'foodstore_u4_arancibia_romero' THEN
        RAISE EXCEPTION 'Usar la copia foodstore_u4_arancibia_romero';
    END IF;
    IF (SELECT count(*) FROM public.pedido) <> 200000
       OR (SELECT max(id) FROM public.pedido) <> 200000
       OR (SELECT count(*) FROM public.detalle_pedido) <> 399997
       OR (SELECT count(*) FROM public.producto) <> 50000
       OR (SELECT count(*) FROM public.usuario) <> 20000 THEN
        RAISE EXCEPTION 'La base no coincide con el respaldo heredado';
    END IF;
    IF to_regclass('public.mv_u4_ventas_categoria_dia') IS NOT NULL THEN
        RAISE EXCEPTION 'La carga del dia debe preceder a la instalacion U4';
    END IF;
END $$;
INSERT INTO public.pedido (usuario_id, fecha, forma_pago)
SELECT 1 + ((g - 1) % 20000),
       (CURRENT_DATE::timestamp AT TIME ZONE 'America/Argentina/Buenos_Aires')
           + (g - 1) * INTERVAL '1 second',
       'EFECTIVO'::public.forma_pago
FROM generate_series(1, 25000) g;
ALTER TABLE public.detalle_pedido DISABLE TRIGGER trg_recalcular_total;
INSERT INTO public.detalle_pedido
    (pedido_id, producto_id, cantidad, precio_unitario)
SELECT p.id, pr.id, 1, pr.precio
FROM public.pedido p
CROSS JOIN generate_series(0, 3) k
JOIN public.producto pr
    ON pr.id = 1 + ((p.id * 17 + k * 7919) % 50000)
WHERE p.id BETWEEN 200001 AND 225000;
UPDATE public.pedido p
SET total = t.total
FROM (
    SELECT pedido_id, SUM(subtotal) AS total
    FROM public.detalle_pedido
    WHERE pedido_id BETWEEN 200001 AND 225000
    GROUP BY pedido_id
) t
WHERE p.id = t.pedido_id;
ALTER TABLE public.detalle_pedido ENABLE TRIGGER trg_recalcular_total;
DO $$
BEGIN
    IF (SELECT count(*) FROM public.pedido) <> 225000
       OR (SELECT count(*) FROM public.detalle_pedido) <> 499997
       OR EXISTS (
            SELECT 1 FROM public.pedido p
            WHERE p.id BETWEEN 200001 AND 225000
              AND p.total <> COALESCE((SELECT SUM(d.subtotal)
                  FROM public.detalle_pedido d WHERE d.pedido_id = p.id), 0)
       ) THEN
        RAISE EXCEPTION 'Fallo de conteo o de totales en la nueva actividad';
    END IF;
END $$;
COMMIT;
REFRESH MATERIALIZED VIEW public.mv_facturacion_categoria_mes;
ANALYZE public.pedido;
ANALYZE public.detalle_pedido;
SELECT 'pedidos_heredados' AS control, count(*) AS filas
FROM public.pedido WHERE id <= 200000
UNION ALL SELECT 'pedidos_agregados_hoy', count(*)
FROM public.pedido WHERE id BETWEEN 200001 AND 225000
UNION ALL SELECT 'detalles_heredados', count(*)
FROM public.detalle_pedido WHERE pedido_id <= 200000
UNION ALL SELECT 'detalles_agregados_hoy', count(*)
FROM public.detalle_pedido WHERE pedido_id BETWEEN 200001 AND 225000;
```